Dynamic Programming Algorithms for Haplotype Blocks Partitioning with TagSNPs Minimization

Yaw-Ling Lin* Tso-Ching Lee† Wen-Pei Chen

Dept. of Comput. Sci. and Info. Management,
College of Computing and Informatics, Providence University,
200 Chung Chi Road, Shalu, Taichung, Taiwan 433.
E-mail: yllin@pu.edu.tw, g9471023@pu.edu.tw

Abstract

Recent studies show that the patterns of linkage disequilibrium (LD) observed in human chromosome reveal a block-like structure; the high LD regions are called *haplotype blocks*. The existence of haplotype block structures has serious implications for association-based methods in mapping of disease genes. *A Single Nucleotide Polymorphism* or SNP is a DNA sequence variation occurring when a single nucleotide in the genome differs between members of species. In this paper, we propose several efficient algorithms for identifying haplotype blocks in the genome. Especially, we develop a dynamic programming algorithm for haplotype block partitioning to minimize the number of tagSNPs required to account for most of the common haplotypes in each block. We implement these algorithms and analyze the chromosome 21 haplotype data given by Patil et al. [14]. As a result, we identify a total of 2,266 blocks (3,260 tagSNPs) which is 45.2% (28.6%) smaller than those identified by Patil et al. or Zhang et al. [18].

Keywords: Diversity, dynamic programming, SNP, haplotype block, tagSNP, haplotype block partition.

1 Introduction

Mutation in DNA is the principle factor resulted in the phenotypic differences among human beings, and SNPs (single nucleotide polymorphism) are the most common mutations, hence it is fundamental to complete a map of all SNPs in the human population. Global pattern of human DNA sequence variation (haplotypes) defined by common SNPs have important implications for identifying disease association and human traits [5, 15]. Recent studies have shown that the patterns of linkage disequilibrium (LD) observed in human chromosome reveal a block-like structure [5, 6, 14], and therefore the entire chromosome can be partitioned into high LD regions interspersed by low LD regions. The high LD regions are called *haplotype blocks* and the low LD ones are referred to as *recombination hotspots*. There is little or even no occurrence of recombination within a haplotype block, and the SNPs are highly correlated in the block. Furthermore, each haplotype block, in which the genome is largely made up of regions of low diversity, can be characterized by a small number of SNPs, which are referred to as *tagSNPs* [10]. This characteristic is very important and useful for medicine or therapy.

Studying on SNP and haplotype blocks not only decrease the cost for detecting inherited diseases but also has many contributions for classifying the race of human and researching on species evolution. Our ultimate goal is to select haplotype block designations that best capture the structure within the data.

Diversity functions

Several operational definitions has been used to identify haplotype-block structures, including LD-based [6, 16], recombination-based [9, 17], information-complexity-based [2, 11, 7] and diversity-based [4, 14, 19] methods. The result of block partition and the meaning of each haplotype block may be different by using different measur-

*This work is supported by grants from the Taichung Veterans General Hospital and Providence University (TCVGH-PU-968110) and in part by the National Science Council (NSC-95-2221-E-126-007) Taichung, Taiwan, Republic of China.

†Address: Taichung Veterans General Hospital. 160, Sec. 3, Taichung Kang Road, Taichung, Taiwan 407. E-mail: tclee@vghtc.gov.tw

ing formula. For simplicity, haplotype samples can be converted into haplotype matrices by assigned major alleles to 0 and minor alleles to 1. Given an $m \times n$ haplotype matrix A, a *block* $A(i, j)$ (i, j are the block boundaries) of matrix A is viewed as m haplotype strings; they are partitioned into groups by merging identical haplotype strings into the same group. The probability p_i of each haplotype pattern s_i, is defined accordingly such that $\sum p_i = 1$. As an example, Li [12] proposes a diversity formula defined by

$$\delta_D(S) = 1 - \sum_{s_i \in S} p_i^2. \quad (1)$$

Note that $\delta_D(S)$ is the probability that two haplotype blocks chosen at random from S are different from each other. Other different diversity functions have been discussed in the literatures [4, 13, 14, 19].

Definition 1 (haplotype block diversity) *Given an interval $[i, j]$ of a haplotype matrix A, a* diversity function*, $\delta : [i, j] \to \delta(i, j) \in \mathbb{R}$ is an evaluation function measuring the diversity of the submatrix $A(i, j)$.*

Diversity measurement usually reflects the activity of recombination events occurred during the evolutionary process. Generally, haplotype blocks with low diversity indicates conserved regions of genome.

Definition 2 (monotonic diversity) *A diversity function δ is said to be* monotonic *if, for any haplotype block (interval) $I = [i, j]$ of A, it follows that $\delta(i', j') \leq \delta(i, j)$ whenever $[i', j'] \subset [i, j]$; that is, the diversity of any subinterval of I is always no larger than the diversity of I.*

It is easily verified that many diversity functions, including the diversity function $\delta_D(S)$ defined by (1), are monotonic. For a diversity-based test, methods can be classified into two categories: those that divide strings of SNPs into blocks on the basis of the decay of LD across block boundaries and those that delineate blocks on the basis of some haplotype-diversity measure within the blocks. Patil et al. [14] defined a haplotype block as a region in which a fraction of percent or more of all the observed haplotypes are represented at least n times or at a given threshold in the sample. They applied the optimization criteria outlined by Zhang et al. [18, 19] and describe a general algorithm that defines block boundaries in a way that minimizes the number of tagSNPs that are required to uniquely distinguish a certain percentage of all the haplotypes in a region. Patil et al. have developed a greedy algorithm and identified a total of 4,563 tagSNPs and a total of 4,135 blocks to define the haplotype structure of human chromosome 21. In each block they required at least 80% of haplotype must be represented more than once in the block. For each block, they used the greedy algorithm to identify the minimum number of tagSNPs that distinguish at least 80% percent of the unambiguous haplotypes in the block. In addition, Zhang et al. [18] used a dynamic programming approach to reduce the numbers of blocks and tagSNPs to 2,575 and 3,582, respectively. In this paper, we propose two dynamic programming algorithms concerning two haplotype block partition problems.

Problem 1 (longest-k-blocks) *Given a haplotype matrix A and a diversity upper limit D, we wish to find k feasible blocks such that the total length is maximized. That is, output the set $S = \{B_1, B_2, \ldots, B_k\}$, with $\delta(B) \leq D$ for each $B \in S$, such that $|B_1| + |B_2| + \cdots + |B_k|$ is maximized.*

In section 2.1, we show that, assuming the given diversity function is monotonic and the given haplotype matrix is preprocessed for finding the farthest sites indices, the longest-k-block problem can be solved by using $O(n)$ space, in $O(kn)$ time.

Problem 2 (longest-blocks-t-tagSNPs) *Given a haplotype matrix A and a diversity upper limit D, we wish to find a list of feasible blocks whose total tagSNP numbers is less than t such that the total length is maximized. That is, output the set $S = \{B_1, B_2, \ldots, B_{|S|}\}$ such that $(\forall B_i \in S)(\delta(B_i) \leq D)$ and $\sum tag(B_i) \leq t$; $tag(B_i)$ denote the number of tagSNPs required for block B_i, so that $|B_1| + |B_2| + \cdots + |B_{|S|}|$ is maximized.*

In section 2.2, we show that, assuming all of the feasible blocks and tagSNPs required for each block have been preprocessed, the longest-blocks-t-tags problem can be solved in $O(tL)$ time, here L denote the total number of feasible blocks. For the same sample used, based on the same criteria adopted by Patil et al., we identify a total of 2,266 blocks, which can be tagged by 3,260 tagSNPs. The number of blocks and tagSNPs we identified are 45.2% and 28.6% less than those identified by Patil et al.. Our results are also slightly better than Zhang et al.'s either in the number of tagSNPs used or the total block numbers.

Note that the definition of the haplotype block diversity evaluation function (δ) we used in this paper is equal to the ratio of singleton haplotypes to unambiguous haplotypes in the blocks. It is also equal to 1 minus the ratio of common haplotypes to unambiguous haplotypes; in other words, the 80% of common haplotyps coverage in Patil et al. is equal to 20% (or 0.2) of haplotype diversity by our definition. That is, we required the diversity of each block ≤ 0.2. We must point out that the δ-function used here is not monotonic.

2 Method

SNP haplotype patterns and disease gene in the same blocks are associative [5, 15], and therefore we can analyze the relation between certain haplotype patterns and disease gene if a chromosome region contains disease gene but no recombination occurred. TagSNPs can capture most of the haplotype dversity in the blocks, and therefore could potentially capture most of the information for association between a trait and the SNP marker loci. We can figure out the diversity and features of each haplotype block easily and economically with using tagSNPs. For these reasons, we want to find the longest haplotype blocks such that the number of tagSNPs is minimized. In this section, we propose two algorithms for partitioning SNP haplotypes into blocks. By the first algorithm, we can find the longest segmentation consists of k feasible blocks in $O(kn)$ time and linear space after the preprocessing of *the left farthest site* $L[i]$ [13] and *the right farthest site* $R[i]$ for each SNP marker i. After partitioning blocks, we select tagSNPs in each block. Using this method we can partition haplotypes into minimum number of blocks with modest size of tagSNPs number. By the second algorithm, we can find the longest segmentation covered by t tagSNPs in $O(tL)$ time after the preprocessing of left good partners L_i for each marker i and tagSNPs required for each feasible block. Using this method we can partition haplotype into minimal number of blocks with minimum number of tagSNPs. Note that these methods can be used for any block diversity measurement.

2.1 A linear space algorithm for haplotype block Partitioning

In our previous study [3], given an $m \times n$ haplotype matrix A and a diversity upper limit D, an $O(nk)$ time dynamic programming algorithm is proposed for finding a maximized segmentation S consists of k feasible monotonic blocks with the diversity of each block $\leq D$. Assume the diversity function is monotonic, the recurrence relation is shown as follow:

$$\begin{aligned} f(k,1,j) &= \max\{f(k,1,j-1), \\ &\qquad f(k-1,1,L[j]-1)+j-L[j]+1\} \end{aligned}$$

The idea behind the recurrence relation is as follow: the k-th block of the maximal segment S in $[1,j]$ either does not include site j; otherwise, the block $[L[j],j]$ must be the last block of S. Note that $f(k,1,j)$ can be determined in $O(1)$ time suppose $f(k-1,1,\cdot)$'s and $f(k,1,1..(j-1))$'s being ready. It follows that $f(k,1,\cdot)$'s can be calculated from $f(k-1,1,\cdot)$'s, totally in $O(n)$ time. Thus a computation ordering from $f(1,1,\cdot)$'s, $f(2,1,\cdot)$'s, ..., to $f(k,1,\cdot)$'s leads to the total of $O(nk)$ time. We can apply the dynamic programming theory to general case and get the lemma 1.

Lemma 1 *Given a submatrix $A'(i,j)$ of $m \times n$ haplotype matrix A and a diversity upper limit D, for all constrained interval $[i,j^*]$, $i \leq j^* \leq j$, find a segmentation consists of k feasible blocks such that the total length is maximized can be done in $O(|j-i|k)$ time after the preprocessed left farthest markers, $L[i]'s$ are prepared.*

Note that finding a segmentation consists of k feasible blocks such that the total length is maximized can be easily calculated by the dynamic programming based on the recurrence relation. However, it is not obvious how we can use the result to retrieve the k intervals using linear space. In order to solve this problem, we can use the concept which is similar to [8]. We find a cut-point x^* to divide n SNP sites into two parts, n_1 and n_2, and such that there are $\lfloor \frac{k}{2} \rfloor$ blocks in the n_1 and $\lceil \frac{k}{2} \rceil$ blocks in the n_2. Here $n_2 = n - n_1$. Therefore, we can get the following recursion relation.

$$f(k,i,j) = f(\lfloor \frac{k}{2} \rfloor, i, x^*) + f(\lceil \frac{k}{2} \rceil, x^*+1, j) \quad (2)$$

While $k = 1$, we can calculate the boundaries of the block by scanning the farthest left marker array, and then append the longest feasible block in $[i,j]$ to a global data structure. The algorithm is shown in Figure 1.

Theorem 1 (longest-k-blocks) *Given a haplotype matrix A and a diversity upper limit D, compute the longest k-block and their boundaries can be done in $O(nk)$ time and using $O(n)$ space after the preprocessed left and right farthest markers, $L[i]'s$ and $R[i]'s$ are prepared.*

```
LIS(k, i, j)        ▷ Lis: List k blocks in [i, j] with maximized total length.
Input: Interval [i, j] and number of blocks k.
Output: k's blocks and their boundaries.
Global variable: L, R, Y, A, B, C.   ▷ L and R are used to store the good partner pointers L[x]'s and R[x]'s
                 which have been preprocessed dependent on diversity constraints D, Y is used to store the
                 results of LIS(k, i, j), A, B, C are global temporary working storages.
1 if k ≤ 1 then
2    Append the boundaries of the longest block in [i, j] to Y
3    return
4 for x ← i to j do        ▷ Initiate the boundary condition of f(k, i, j).
5      C[x] ← 0
6 for y ← 1 to ⌊k/2⌋ do        ▷ Compute A[x] = f(⌊k/2⌋, i, x), ∀x ∈ [i ... j − 1].
7      for x ← i to j − 1 do
8          if x = i then
9              temp1 ← 1
10         else
11             temp1 ← A[x − 1]
12         if L[x] ≤ i then        ▷ L[x] ∉ [i, j], exceeding the boundary region.
13             temp2 ← x − i + 1
14         else
15             temp2 ← C[L[x] − 1] + x − L[x] + 1
16         A[x] ← max{temp1, temp2}
17     copy A[x] to C[x] for next iteration of x        ▷ C[x] is used to store the temporary results of f(k, i, j).
18 for x ← i to j do        ▷ Initiate the boundary condition of f(k, i, j).
19     C[x] ← 0
20 for y ← 1 to ⌈k/2⌉ do        ▷ Compute B[x] = f(⌈k/2⌉, x + 1, j), ∀x ∈ [i ... j − 1].
21     for x ← j downto i + 1 do
22         if x = j then
23             temp1 ← 1
24         else
25             temp1 ← A[x + 1]
26         if R[x] ≥ j then        ▷ R[x] ∉ [i, j], exceeding the boundary region.
27             temp2 ← j − x + 1
28         else
29             temp2 ← C[R[x] + 1] + R[x] − x + 1
30         B[x] ← max{temp1, temp2}
31     copy B[x] to C[x] for next iteration of x        ▷ C[x] is used to store the temporary results of f(k, i, j).
32 temp ← −∞
33 for x ← i to j − 1 do        ▷ Find x* = arg max_{i≤x≤j}{A[x] + B[x]}.
34     if (A[x] + B[x + 1]) > temp then
35         x* ← x
36         temp ← (A[x] + B[x + 1])
37 LIS(⌊k/2⌋, i, x*)        ▷ recursive call.
38 LIS(⌈k/2⌉, x* + 1, j)        ▷ recursive call.
```

Figure 1: The $O(nk)$ time and linear space algorithm for haplotype blocking.

Proof. We propose an $O(nk)$ time algorithm, LIS(k, i, j), shown in Figure 1. Note that $O(mn)$ time suffices to preprocess to find farthest right markers $R[i]'s$ and farthest left markers $L[i]'s$ for each marker site i as shown in [13]. The correctness of the algorithm can be shown as follow. When $k = 1$, the algorithm just scan the farthest left marker array and append the longest feasible sequence in $[i, j]$ to global data structure Y-list. If $k > 1$, we must find a cut-point x^* between site i and site j such that there are $\lfloor \frac{k}{2} \rfloor$ blocks in the left hand side of x^* and $\lceil \frac{k}{2} \rceil$ blocks in the right hand side of x^*, and furthermore the total length of blocks in the left hand side and right hand side of x^* must be maximized (*i.e.* Line 4-36). In the case of $k > 1$, we first compute $f(\lfloor \frac{k}{2} \rfloor, i, x)'s$ and $f(\lceil \frac{k}{2} \rceil, x+1, j)'s$, for all $x = i \sim j$, and put results into A array and B array. Then, we find a x^* such that the total length of blocks in the left hand side and right hand side of x^* is maximized. That is, find a x^* such that $f(\lfloor \frac{k}{2} \rfloor, i, x^*)+f(\lceil \frac{k}{2} \rceil, x^* + 1, j)$ is maximized. Next steps we use recursive algorithm LIS$(\lfloor \frac{k}{2} \rfloor, i, x^*)$ and LIS$(\lceil \frac{k}{2} \rceil, x^* + 1, j)$ to list $\lfloor \frac{k}{2} \rfloor$ blocks in $[i, x^*]$ and $\lceil \frac{k}{2} \rceil$ blocks in $[x^* + 1, j]$.

In the algorithm, we use six global data structures involving arrays L, R, A, B, C and Y-list. L array and R array are used to store the good partner points $L[i]$'s and $R[i]$'s which have been calculated in preprocessing. Y-list is used to store the boundaries of k blocks. In addition, we use A array and B array to store the results of $f(\lfloor \frac{k}{2} \rfloor, i, x)'s$ and $f(\lceil \frac{k}{2} \rceil, x + 1, j)'s$. During the computation of $f(\lfloor \frac{k}{2} \rfloor, i, x)'s$ and $f(\lceil \frac{k}{2} \rceil, x+1, j)'s$, we use a C array replacing a $k \times n$ table to store the temporary results that will be used to calculate further results. All the space of R, L, A, B and C array are n. The space of Y-list is k, $k \leq n$ in general case, so the space used by the algorithm is $O(n)$.

The time complexity of the algorithm is $O(nk)$ as shown in the following by induction. Let $T(n, k)$ denote the time needed for LIS$(k, 1, n)$. Assume that $T(n', k') \leq c_2 n' k'$ for all $n' < n$, $k' < k$. According to the algorithm, we have:

$$T(n,k) = \underbrace{c_1 nk}_{\text{line 4-36}} + \underbrace{T(n_1, \lfloor \frac{k}{2} \rfloor)}_{\text{line 37}} + \underbrace{T(n - n_1, \lceil \frac{k}{2} \rceil)}_{\text{line 38}}$$

By induction,

$$\begin{aligned} T(n,k) &\leq c_1 nk + c_2 n_1 \lfloor \frac{k}{2} \rfloor + c_2 (n - n_1) \lceil \frac{k}{2} \rceil \\ &\leq (c_1 k + c_2 \lceil \frac{k}{2} \rceil) n + c_2 n_1 \lfloor \frac{k}{2} \rfloor - c_2 n_1 \lceil \frac{k}{2} \rceil \\ &\leq (c_1 k + c_2 \lceil \frac{k}{2} \rceil) n \\ &\leq (c_1 + c_2 \frac{2}{3}) kn \text{ (when } k \geq 2, \lceil \tfrac{k}{2} \rceil \leq \tfrac{2}{3} k) \\ &\leq c_2 nk \end{aligned}$$

Let $c_2 = 3c_1$, the above inequality will come into existence, so we can prove the time complexity of the algorithm is $O(nk)$. □

Although we assume that the block diversity evaluation function we used here is monotonic, we can modify small part of the algorithm such that it can be apply to non-monotonic blocks. In the case of non-monotonic blocks, for each SNP markers i, we use L_i to denote the set of all x such that $[x, i]$ is a feasible haplotype block. Let $L = nl = \sum_{i=1}^{n} |L_i|$, l is the average number of $|L_i|$ for each marker i. It can be shown that the modified algorithm spends $O(knl)$ time and $O(nl)$ space. By a similar proof of argument as shown above, the correctness of the algorithm can also be shown.

The experimental results of the algorithm for finding the maximized segmentation S consists of k feasible blocks based on the specific diversity threshold D have been shown in [3]. Due to the space constraints, our system crashes when the size of genome becomes too long. By using the result of this section, the system space constraints is resolved. The system now can handle an input size of 50Mb regardless any choices of k. The system has been fully tested and executed reliably. The interested reader can obtain the developed system on our web site [1].

2.2 Longest blocks partition using limited number of tagSNPs

In this subsection, we show a dynamic programming algorithm to partition haplotype blocks with constraints on diversity and tagSNPs number. That is, we want to find the longest segmentation S containing blocks with the diversity of each block is less than D and the total tagSNPs number required for these blocks does not exceed a specific number t. The problem definition is shown in Problem 2. According to the haplotype block definition in Patil et al. [14], we know that the common haplotypes coverage evaluation function is not monotonic. That is, for each SNP marker j there will be a left farthest marker i so that $[i, j]$ is the longest haplotype block among all blocks that terminated at site j, but some interval $[i', j] \subset [i, j]$ are not feasible blocks. Thus, before the computation, we need to preprocess the

MaxBlock(n, T) ▷ Find a segmentation S consists of k feasible blocks in $[1, n]$, with the diversity of each block $< D$, and the total of tagSNP $\leq T$ such that the length of S is maximized.
Input: n: the length of haplotype matrix A; T: tagSNP number used.
▷ Every L_i for each marker site and tagSNPs required for each block have been preprocessed.
Output: The length of the longest segmentation in $[1, n]$.
Notation: The two dimensional array $length[t, i]$ represent the $f(i, t)$ function listed in (3).

```
1 if tag(1,1) = 0 then        ▷ Initiate the boundary condition.
2    length[0,1] ← 1
3 else
4    length[0,1] ← 0
5 for t ← 1 to T do
6      length[t,1] ← 1

1 for t ← 0 to T do           ▷ Calculate f(i,t) listed in (3).
2      for i ← 2 to n do
3           temp ← length[t,i−1]      ▷ The last block of segmentation S does not include site i.
4           for each k ∈ L_i do
5                 if t ≥ tag(k,i) and (i−k+1)+length[t−tag(k,i),k−1] > temp then
6                    temp ← (i−k+1)+length[t−tag(k,i),k−1]
7           length[t,i] ← temp
8 return length[t,n]
```

Figure 2: The dynamic programming algorithm for longest blocks partition with constraints on diversity and tagSNPs number.

set of *left good partners* L_i for each SNP marker i, $L_i = \{x|[x, i]$ is a feasible haplotype block$\}$. Furthermore, we assume that the number of tagSNPs required for each feasible haplotype block is also precomputed. After the preprocessing, we can show that finding the longest blocks covered by t tagSNPs can be found in $O(tL)$(or $O(tnl)$); here t denote the number of tagSNPs used, and $L = \sum_{i=1}^{n} |L_i|$ denote the total number of feasible blocks.

Let $f(i, t)$ define the length of the longest segmentation of haplotype $A(1, i)$ covered by t tagSNPs, and $tag(i, j)$ denote the number of tagSNPs required for block which bounded by sites i and j. It is interesting to note that $f(i, t)$ can be computed by the following recurrence:

$$
\begin{aligned}
f(1,0) &= \begin{cases} 0 & \text{if } tag(1,1) > 0 \\ 1 & \text{if } tag(1,1) = 0 \end{cases} \\
f(1,t) &= 1 \text{ if } t \geq 1 \\
f(i,t) &= -\infty \text{ if } t < 0
\end{aligned}
$$

$$
\text{f(i,t)} = \max \left\{ \begin{array}{l} f(i-1,t) \\ \max_{k \in L_i} \left\{ \begin{array}{l} (i-k+1)+ \\ f(k-1, t-tag(k,j)) \end{array} \right\} \end{array} \right. \quad (3)
$$

The idea behind the recurrence relation is illustrated at Figure 3. The maximized segmentation S between site 1 and site i will have two cases, either the site i is included in the last block of S or not. If site i is not included in the last block of S, it will find S between site 1 and site $i-1$, otherwise there will exist a site $k \in L_i$ such that $[k, i]$ is the last block of S. In the latter case, the tagSNPs required for bock $[k, i]$ is $tag(k, i)$ which has been calculated in preprocessing, so we can find other blocks which covered by other $t-tag(k, i)$ tagSNPs between site 1 and site $k-1$.

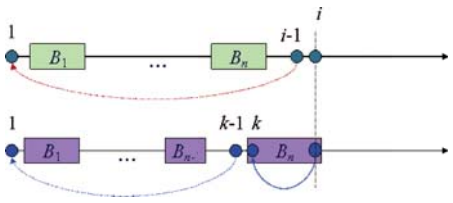

Figure 3: Illustration of the ideas of recurrence $f(i, t)$.

Note that if l is the average number of $|L_i|$ for each marker i, $f(i, t)$ will can be determined in $O(l)$ time suppose $f(1..(i-1), t)$'s and $f(\cdot, 1..(t-1))$'s being ready. It follows that $f(\cdot, t)$'s can be calculated from $f(\cdot, 1..(t-1))$'s totally in $O(nl)$ time. Thus a computation ordering from $f(\cdot, 1)$'s, $f(\cdot, 2)$'s, ..., to $f(\cdot, t)$'s leads to the following result.

Common SNPs/block	No. of block	Length	Avg. length	All blocks(%)	Common SNPs(%)
> 10	736	18,703	25.41	32.48	77.78
$3 - 10$	751	4,338	5.78	33.14	18.04
< 3	779	1,006	1.29	34.38	4.18
Total	2,266	24,047	10.61	100.00	100.00
Coverage: 80%; 3,260 tagSNPs; 2,266 blocks; max. block: 128					
Zhang et al: 3,582 tagSNPs, 2,575 blocks; Patil et al: 4,563 tagSNPs, 4,135 blocks					

Table 1: Blocks partitioning of chromosome 21 with 80% coverage (3,260 tags required).

genome region covered (%)	tagSNPs required	extra tagSNPs required	0-tagged blocks number	blocks with tags number > 0	avg. length of non-0-tagged blocks
38.55	0	0	6136	0	1.51(0-tagged blocks)
40	8	8	6111	6	67.17
50	127	119	5630	80	43.75
60	327	200	4991	193	35.39
70	623	296	4213	347	30.22
80	1045	422	3307	567	25.14
90	1709	664	2208	888	20.58
100	3260	1551	631	1635	14.10

Table 2: The analysis data based on genome region covered.

Theorem 2 (longest-blocks-t-tagSNPs) *Given a haplotype matrix A, a diversity upper limit D and the number of tagSNP t, find a segmentation S consists of k feasible blocks such that $(\forall_i)(\delta(B_i) \leq D)$ and $\sum tag(B_i) \leq T$, so that the total length of S is maximized can be done in $O(tnl)$ time after the preprocessing of L_i and $tag(k, i)$, $k \in L_i$, for each SNP marker i.*

Our dynamic programming algorithm is shown in Figure 2.

3 Experimental results

We apply our dynamic programming algorithm which finds the longest segmentation covered by the specific number of tagSNPs to the haplotype data for chromosome 21 provided by Patil et al. [14]. The data contain 20 haplotype samples and each contains 24,047 SNPs spanning 32.4 Mb of chromosome 21. The minor allele frequency at each marker locus is at least 10%. Using our algorithm with the same criteria as in Patil et al. with coverage of common hpalotypes in the blocks $\geq$ 80%, a total of 3,260 tagSNPs and a total of 2,266 haplotype blocks are identified. In contrast, Patil et al. [14] identified a total of 4,563 tagSNPs and a total of 4,135 blocks and Zhang et al. [18] identified a total of 3,582 tagSNPs and a total of 2,575 blocks. Our dynamic programming algorithm reduces the number of tagSNPs and blocks by 28.6% and 45.2% comparing to Patil et al.. We also demonstrate that the results of Zhang et al. are not optimum.

The properties of blocks we identified are showed in Table 1. Our program discovers a total of 736 blocks containing more than 10 SNPs per block. The blocks with more than 10 SNPs account for 32.48% of all of blocks. The average number of SNPs for all of the blocks is 10.61. The largest block contains 128 common SNPs, which is longer than the largest block (containing 114 SNPs) identified by Patil et al. and the same as in Zhang et al.. Tables 2 and 3 show more analysis data of our experimental results. According to our experimental results, we can partition 38.55 percent of genome region into blocks which do not require any tagSNPs. This is because that most of these blocks just contain few common SNPs, and there are 80 percent of unambiguous haplotypes have the same haplotype pattern (compatible) in these blocks. We termed these SNP loci as *non-informative* markers because they are the same among most (80%) of population. These data also show that as length of the genome region covered increase, we need to increase more and more extra tagSNPs to capture the haplotype information of the blocks, and the number of zero-tagged blocks

tagSNPs used	genome region covered (%)	extra genome region increased (%)	0-tagged blocks	blocks with tags number > 0	avg. length of non-0-tagged blocks
0% (0)	38.55	38.55	6136	0	1.51(0-tagged blocks)
10% (326)	59.99	21.44	4991	192	35.52
20% (652)	70.85	10.86	4145	367	29.37
30% (978)	78.62	7.77	3387	516	26.79
40% (1304)	84.61	5.99	2897	712	22.38
50% (1630)	89.02	4.41	2250	844	21.29
60% (1956)	92.59	3.57	1814	1002	19.41
70% (2282)	95.30	2.71	1478	1159	17.79
80% (2608)	97.29	1.99	1014	1289	16.90
90% (2934)	98.64	1.35	719	1421	15.89
100% (3260)	100.00	1.36	631	1635	14.10

Table 3: The analysis data based on tagSNP required.

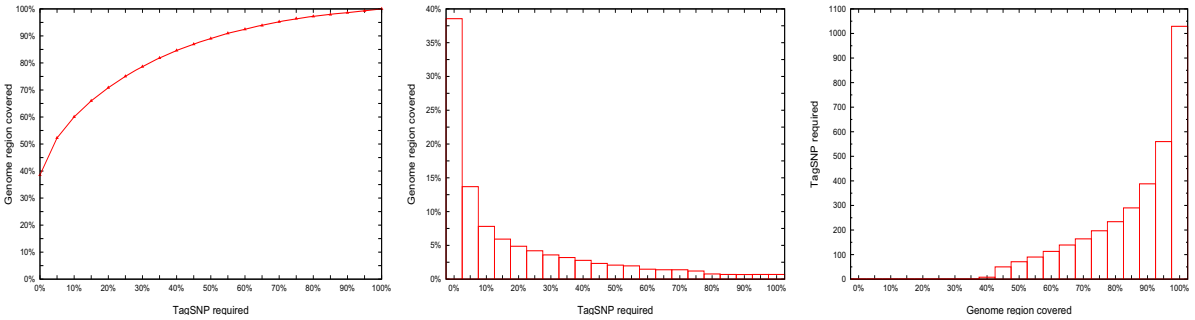

Figure 4: (a) The percentage of genome region covered by the percentage of tagSNP number,(b) the percentage of genome region covered increased while the number of tagSNP number increased by 5 percent and (c) the number of tagSNPs need to increase while the genome region covered increase by 5 percent.

becomes fewer. Note that although the average length of non-zero-tagged blocks become shorter as the genome region covered increase, the average length of total blocks becomes longer.

Figure 4-a shows the percentage of tagSNPs we identified when blocks cover certain percent of genome region. According to experimental results, when blocks cover 70 percent of genome region, we just required 19.1% of tagSNPs (about 623 tagSNPs) to capture the majority of information about haplotypes. This also indicates that our method discovers that only a few tagSNPs is needed to capture the most of genome region information. Figure 4-b shows the percentage of genome region covered increases while the tagSNPs we identified increase by 5 percent. Note that as the number of tagSNPs increase, the marginal percentage of genome region covered decreases. This indicates that, as the genome region covered increases, fewer common SNPs are covered by each tagSNP on average. Figure 4-c shows the number of tagSNPs need to increases, while the percentage of genome region covered increases by 5 percent. We find that as the genome region covered increases much more tagSNPs is needed to capture the haplotypes information. Especially, when the genome region covered increases form 95% to 100%, we need to use another extra 1,014 tagSNPs, about 31.1% of the total tagSNPs. It is interesting to note that our method discovers the marginal utility of tagSNPs decreases as the genome region covered increases.

Furthermore, we examine the influence of common haplotype coverage, ρ, on the block patterns. The coverage with 70%, 80%, and 90% are examined. When the required coverage is 90%, the total number of blocks increases to 3,184. The total number of tagSNPs required to distinguish these blocks increases to 6,917. The length of the largest block decreases to 92 SNPs. These results are also better than Zhang et al.'s (3,573 blocks and 7,536 tagSNPs required). When the coverage is decreased to 70%, the total number of blocks decreases to 2,105 with the largest block contain-

Common SNPs/block	No. of block	Length	Avg. length	All blocks(%)	Common SNPs(%)
> 10	715	14,022	19.61	22.46	58.31
$3 - 10$	1,512	8,657	5.73	47.49	36.00
< 3	957	1,368	1.43	30.05	5.69
Total	3,184	24,047	7.55	100.00	100.00
Coverage: 90%; 6,917 tagSNPs; 3,184 blocks; max. block: 92					
Zhang et al: 7,536 tagSNPs, 3,573 blocks					

Table 4: Blocks partitioning of chromosome 21 with 90% coverage (6,802 tags required).

Common SNPs/block	No. of block	Length	Avg. length	All blocks(%)	Common SNPs(%)
> 10	636	19,673	30.93	30.21	81.81
$3 - 10$	590	3,196	5.42	28.03	13.29
< 3	879	1,178	1.34	41.76	4.90
Total	2,105	24,047	11.42	100.00	100.00
Coverage: 70%; 1,812 tagSNPs; 2,105 blocks; max. block: 199					
Zhang et al: 1,977 tagSNPs, 2,381 blocks					

Table 5: Blocks partitioning of chromosome 21 with 70% coverage (1,848 tags required).

ing 199 common SNPs, and the total number of tagSNPs required to distinguish these blocks decreases to 1,812. The blocks number will decrease to 1495 if we require 1,977 tagSNPs which is the same as in Zhang et al. [18]. According to our experimental results, when the common haplotype coverage of blocks increases, the length of the block becomes shorter, and the number of blocks and tagSNPs required become more. The properties of the blocks for 90% and 70% of coverage are given in Tables 4 and 5, respectively. Some of our primary results have been incorporated into our web-based system, and the system is accessible at `http://bioinfo.cs.pu.edu.tw/~hap/lbpcdt.html`.

4 Conclusion

In this paper, we present dynamic programming algorithms for haplotype blocks partitioning such that the total blocks length is maximized and the total tagSNPs required is minimized. We also show in Theorem 1 that finding longest k-block segmentation with diversity constraints can be done in $O(nk)$ time and $O(n)$ space. In Theorem 2, we show that finding a maximum segmentation with constraints on diversity and tagSNPs number can be done in $O(tnl)$ time.

Compared with Patil et al.'s results, our method identifies longer blocks and the numbers of blocks and tagSNPs required is reduced by 45.2% and 28.6% for the haplotype data on chromosome 21. We also show that the results discovered by our method is superior to Zhang et al.'s [18]. Our method discovers that we just require a few tagSNPs to capture a large portion of genome region information.

Instead of genotyping all of the SNP markers on the chromosome, one may wish to use only the genotype information on the tagSNP. Only about 13.6% (3,260) of all of the SNPs (24,047) can account for 80% of the common haplotypes in each block. This also means that we can figure out the haplotype features of most population by just checking a few SNP markers. Thus, studying the tagSNPs can dramatically reduce the time and effort for genotyping, without losing much haplotype information.

SNP is the most common DNA mutation that causes the phenotypic differences among human beings. The SNP number accounts for 0.74% (24,047) of the total length of human chromosome 21 (32.4 Mb). Using the characteristic of tagSNPs, we show that 3,260 tagSNPs suffice to capture most of information about haplotypes on human chromosome 21. We are tempted to say that the compression ratios of the chromosome to the haplotype, and the haplotype to tagSNPs, are about 1,400 and 7.38.

References

[1] Providence university SNP and haplotype research center. `http://bioinfo.cs.pu.edu.tw/hap/`.

[2] Eric C. Anderson and John Novembre. Finding haplotype block boundaries by using the minimum-description-length principle. *Am. J. of Human Genetics*, 73:336–354, 2003.

[3] Wen-Pei Chen, Tso-Ching Lee, and Yaw-Ling Lin. Haplotype block partitioning and tagSNP selection on human chromosome 21. In *Proceedings of the International Computer Symposium 2006*, pages 1278–1283, Taipei,Taiwan, Dec 04-06, 2006.

[4] D. Clayton. Choosing a set of haplotype tagging SNPs from a larger set of diallelic loci. *Nature Genetics*, 29(2), 2001.

[5] M. J. Daly, J. D. Rioux, S. F. Schafiner, T. J. Hudson, and E. S. Lander. High-resolution haplotype structure in the human genome. *Nature Genetics*, 29:229–232, 2001.

[6] S. B. Gabriel, S. F. Schaffner, H. Nguyen, et al. The structure of haplotype blocks in the human genome. *Science*, 296(5576):2225–2229, 2002.

[7] G. Greenspan and D. Geiger. Model-based inference of haplotype block variation. In *Seventh Annual International Conference on Computational Molecular Biology (RECOMB)*, 2003.

[8] D. S. Hirschberg. A linear space algorithm for computing maximal common subsequences. *Commun. ACM*, 18(6):341–343, 1975.

[9] R. R. Hudson and N. L. Kaplan. Statistical properties of the number of recombination events in the history of a sample of dna sequences. *Genetics*, 111:147–164, 1985.

[10] G. C. Johnson, L. Esposito, B. J. Barratt, et al. Haplotype tagging for the identification of common disease genes. *Nat Genet.*, 29(2):233 – 7, Oct 2001.

[11] M. Koivisto, M. Perola, R. Varilo, W. Hennah, J. Ekelund, M. Lukk, L. Peltonen, E. Ukkonen, and H. Mannila. An mdl method for finding haplotype blocks and for estimating the strength of haplotype block boundaries. In *8th Pacific Symposium on Biocomputing (PSB)*, pages 502–513, 2003.

[12] W.H. Li and D. Graur. *Fundamentals of Molecular Evolution*. Sinauer Associates, Inc, 1991.

[13] Yaw-Ling Lin and Wei-Shun Su. Identifying long haplotype blocks with low diversity. In *Proceedings of the 23rd Workshop on Combinatorial Mathematics and Computation Theory*, pages 151–159, Changhua,Taiwan, Apr 28-29, 2006.

[14] N. Patil, A. J. Berno, D. A. Hinds, et al. Blocks of limited haplotype diversity revealed by high resolution scanning of human chromosome 21. *Science*, 294:1719–1723, 2001.

[15] J. D. Rioux, M. J. Daly, M. S. Silverberg, K. Lindblad, H. Steinhart, Z. Cohen, et al. Genetic variation in the 5q31 cytokine gene cluster confers susceptibility to crohn disease. *Nature Genetics*, 29:223–228, 2001.

[16] J.D. Wall and J.K Pritchard. Haplotype blocks and linkage disequilibrium in the human genome. *Nature Reviews Genetics*, 4(8):587–597, 2003.

[17] N. Wang, J.M. Akey, K. Zhang, R. Chakraborty, and L. Jin. Distribution of recombination crossovers and the origin of haplotype blocks: the interplay of population history, recombination, and mutation. *Am. J. Human Genetics*, 71:1227–1234, 2002.

[18] K. Zhang, M. Deng, T. Chen, M.S. Waterman, and F. Sun. A dynamic programming algorithm for haplotype block partitioning. In *The National Academy of Sciences*, volume 99, pages 7335–7339, 2002.

[19] K. Zhang, Z.S. Qin, J.S. Liu, T. Chen T, M.S. Waterman, and F. Sun. Haplotype block partitioning and tag SNP selection using genotype data and their applications to association studies. *Genome Res.*, 14(5):908–916, 2004.